```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company: MIT
// Engineer: Ahmad Taka
//
// Create Date: 10/07/2021 02:09:50 AM
// Design Name: CRC
// Module Name: crc
// Project Name: CRC
// Target Devices: Artrix-7
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module crc(
    input wire clock,
    input wire start,
    input wire data,
    output logic done,
    output logic [15:0] r
    );
    logic [5:0] bit_counter; //Counts to 48 bits
    always_ff @(posedge clock) begin
        if (start) begin
            //Initial Trigger
            r <= 16'hFFFF;
            done <= 1'b0;
            bit_counter <= 1'd0;
        end else begin
            if (bit_counter == 6'd48) begin
            //Termination
                done <= 1'b1;
                bit_counter <= 1'd0;
            end else begin
            //FSM LOGIC
                bit_counter <= bit_counter + 1'd1;
                r[0] <= data + r[15];
                r[1] <= r[0];
                r[2] <= r[15] + r[1] + data;
                for(int i = 3; i < 15; i++) begin r[i] <= r[i - 1]; end //All register
s 3-14 are equal to previous
                r[15] <= r[14] + r[15] + data;
             end
        end
    end
endmodule



`timescale 1ns / 1ps

//////////////////////////////////////////////////////////////////////////////////
//
// lpset CRC test bench  9/25/2019
//
//////////////////////////////////////////////////////////////////////////////////
```

```
module crc_tb;

   // Inputs
   logic clock;
   logic data_clk;
   logic start;
   logic data;

   // Outputs
   logic done;
   logic [15:0] r;

   // Instantiate the Unit Under Test (UUT)
   crc uut (
      .clock(clock),
      .start(start),
      .data(data),
      .done(done),
      .r(r)
   );


 // this is the input data
   logic [47:0] input_data = 48'h03_01_02_03_30_3A;
   integer i;       // required for "for" loop

   initial begin    // system clock
      forever #5 clock = !clock;
      end

   initial begin    // data_clk, ensures setup time met
      #2
      forever #5 data_clk = !data_clk;
      end

   initial begin
      // Initialize Inputs
      clock = 0;
      data_clk = 0;
      start = 0;
      data = 0;

      // Wait 100 ns for global reset to finish
      #100;

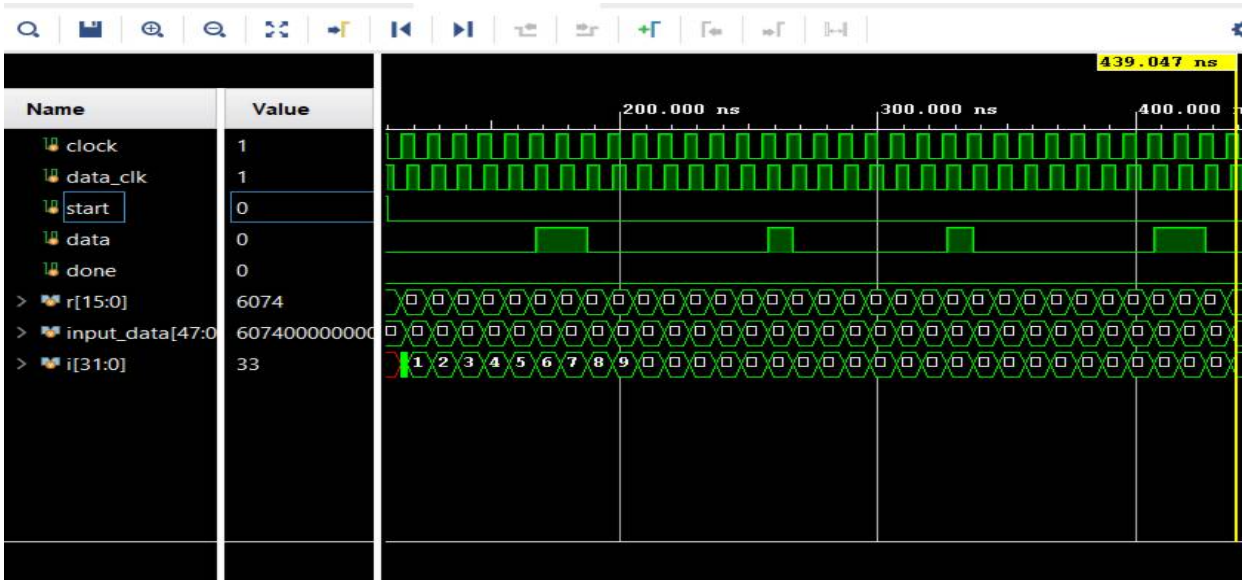
      // Add stimulus here
      start=1;
      #10 start = 0;
      #5;        // change if necessary for your design

      //forever #5 data_clk = !data_clk;
       for (i=0; i<50; i=i+1)
       begin
         data = input_data[47];
         // at each clock, left shift the data
         // note syntax for test bench "for" loop - no "always"
         // note the blocking assignment (immediate)
         @(posedge data_clk) input_data = {input_data[46:0],1'b0};
       end

   $stop; // Pause simulation
   end

endmodule
```

TB Screenshot (start to 32):



TB Screenshot (32 to 48):

